



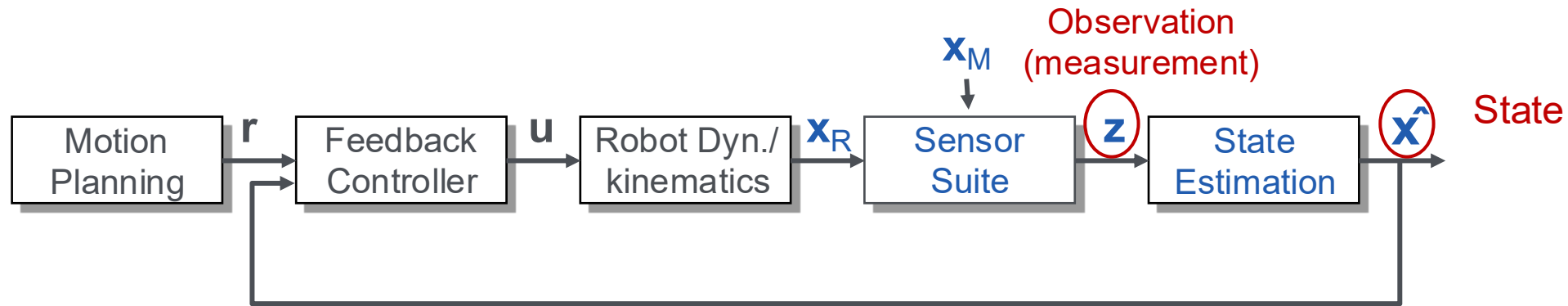
Autonomous Mobile Robots

E06: Nonlinear Least Squares (NLSQ) Optimization

Stefan Leutenegger, Tianyi Zhang, Cédric Le Gentil, Simon Boche



Multi-Sensor Estimation



NLSQ optimization

1. Analytical solution for Linear least square
2. Iterative solution for Non-linear Least Square
3. Use of robust loss functions to handle outliers

Linearising Vector-valued Functions

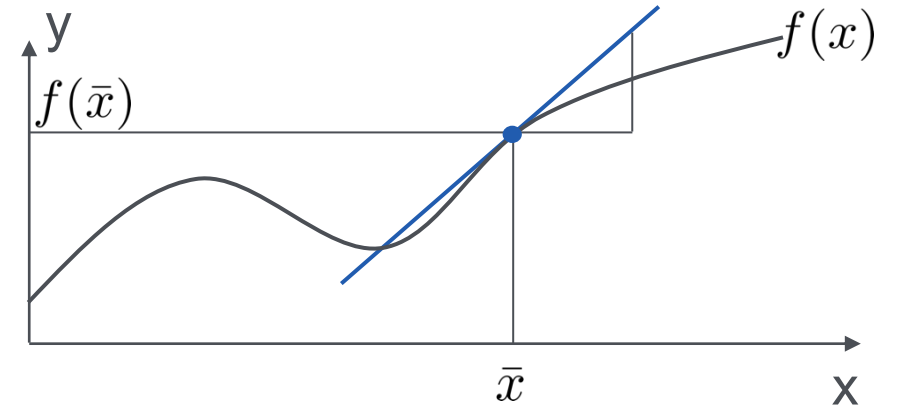
What does linearization mean?

1D case:

We approximate a function $f(x)$ around a **linearisation point** $\bar{x}$ with

$$f(x) \approx f(\bar{x}) + \left. \frac{df}{dx} \right|_{x=\bar{x}} (x - \bar{x}).$$

(First order Taylor approximation)

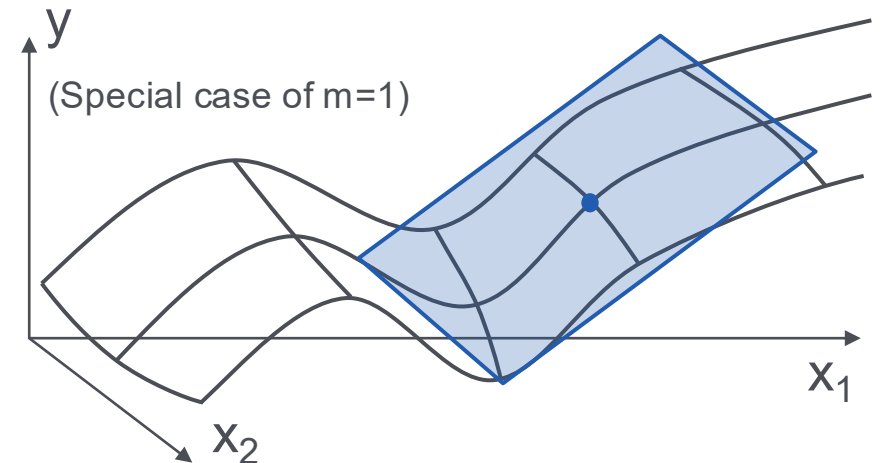


Multivariate case:

$$\mathbf{f}(\mathbf{x}) \in \mathbb{R}^m, \mathbf{x} \in \mathbb{R}^n,$$

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{f}(\bar{\mathbf{x}}) + \mathbf{F}|_{\mathbf{x}=\bar{\mathbf{x}}} (\mathbf{x} - \bar{\mathbf{x}}).$$

Where $\mathbf{F}|_{\mathbf{x}=\bar{\mathbf{x}}} \in \mathbb{R}^{m \times n}$ is called the **Jacobian Matrix**.



The Jacobian Matrix

Here is how the Jacobian is defined:

$$\mathbf{F} = \frac{d\mathbf{f}}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

Note: for the case of a scalar field, i.e.

$f(\mathbf{x}) \in \mathbb{R}, \mathbf{x} \in \mathbb{R}^n$, this corresponds to the gradient vector.

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} 4x_1 - 3x_2^2 \\ x_2 + 2x_3^3 \end{bmatrix}.$$

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} 4 & -6x_2 & 0 \\ 0 & 1 & 6x_3^2 \end{bmatrix}.$$



Carl Gustav Jacobi Jacobi
1804-1851

en.wikipedia.org/wiki/Carl_Gustav_Jacob_Jacobi

Linear Least Squares & Normal Equations

Given:

- N data points $\tilde{z}_i$ at t_i .
- Assume the data can be explained with model $z = (a_1 t + a_0)$.

Find:

$\mathbf{x}^* = [a_0^*, a_1^*]^T$ minimising the sum of square differences

$$c(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \|\tilde{z}_i - (a_1 t_i + a_0)\|_2^2.$$

Solution:

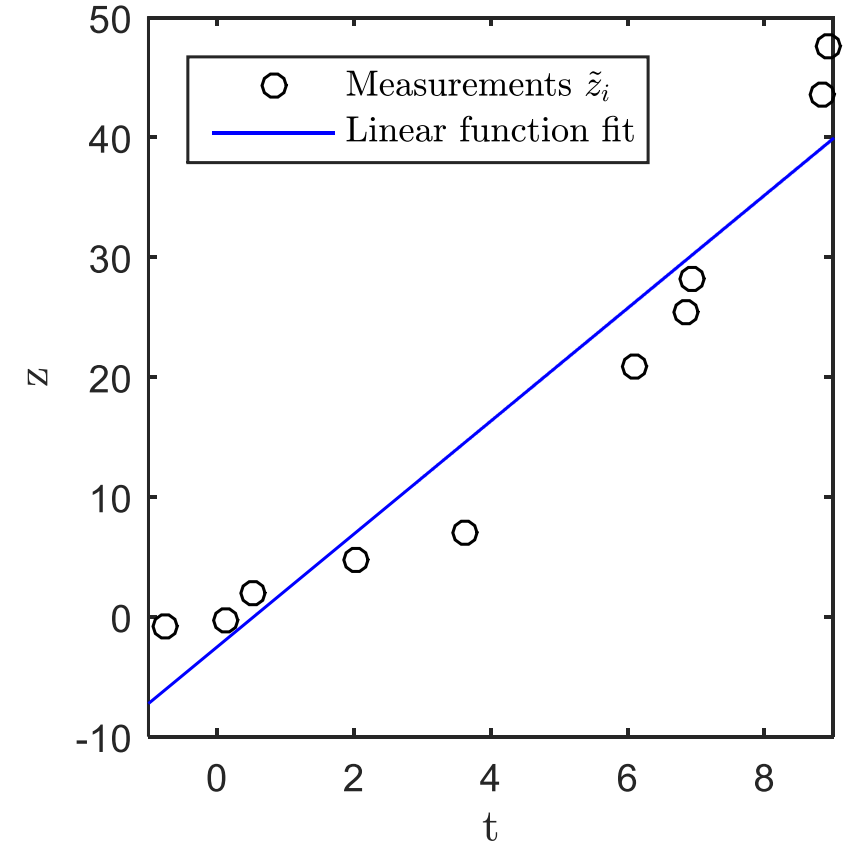
Re-write as $c(\mathbf{x}) = \frac{1}{2}(\tilde{\mathbf{z}} - \mathbf{H}\mathbf{x})^T(\tilde{\mathbf{z}} - \mathbf{H}\mathbf{x})$,

$$\tilde{\mathbf{z}} = \begin{bmatrix} \tilde{z}_1 \\ \vdots \\ \tilde{z}_N \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & t_1 \\ \vdots & \vdots \\ 1 & t_N \end{bmatrix}.$$

Necessary condition for optimality: $\left. \frac{\partial}{\partial \mathbf{x}} c(\mathbf{x}) \right|_{\mathbf{x}^*} = \mathbf{0}$.

$$\mathbf{H}^T(\tilde{\mathbf{z}} - \mathbf{H}\mathbf{x}^*) = \mathbf{0} \implies \boxed{\mathbf{x}^* = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \tilde{\mathbf{z}}.}$$

Normal Equations



The Gauss-Newton Method

Given:

- Nonlinear error terms of the form $\mathbf{e}_i(\mathbf{x}) := \tilde{\mathbf{z}}_i - \mathbf{h}_i(\mathbf{x}) \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_i)^*$.

Iterative solution to the associated nonlinear least squares problem:

- Pick a suitable **starting point** $\bar{\mathbf{x}}^0$.

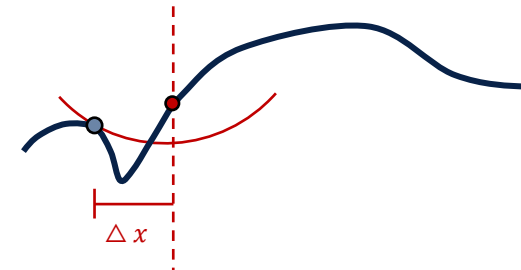
$$\min_x \sum_{i=1}^N e_i(x)^T e_i(x) \Rightarrow \min_{\Delta x} \sum_{i=1}^N e_i(x^k + \Delta x)^T e_i(x^k + \Delta x)$$

- Approximate the cost by employing linear error terms $\mathbf{e}_i(\mathbf{x}) \approx \mathbf{e}_i(\bar{\mathbf{x}}^k) + \mathbf{E}_i(\bar{\mathbf{x}}^k) \Delta \mathbf{x}$, with $\mathbf{E}_i(\bar{\mathbf{x}}^k) = \frac{\partial}{\partial \mathbf{x}} \mathbf{e}_i(\mathbf{x})|_{\bar{\mathbf{x}}^k}$ (note: typically sparse) and formulate the **normal equations**:

$$\underbrace{\sum_i \mathbf{E}_i(\bar{\mathbf{x}}^k)^T \mathbf{R}_i^{-1} \mathbf{E}_i(\bar{\mathbf{x}}^k)}_{=: \mathbf{A}} \Delta \mathbf{x} = - \underbrace{\sum_i \mathbf{E}_i(\bar{\mathbf{x}}^k)^T \mathbf{R}_i^{-1} \mathbf{e}_i(\bar{\mathbf{x}}^k)}_{=: \mathbf{b}}.$$

- Solve** the normal equations for $\Delta \mathbf{x}$ using e.g. **Cholesky Decomposition**
- Update** the current estimate as $\bar{\mathbf{x}}^{k+1} = \bar{\mathbf{x}}^k + \Delta \mathbf{x}$.
- Check **convergence**: if not OK, go to step 2.

*) Note: a prior can be added to the error terms analogously.



Levenberg-Marquardt

A trust-region method that interpolates between gradient-descent and Gauss-Newton.

1. Pick a **starting point** $\bar{\mathbf{x}}^0$, initial parameter $\lambda^0 = \max \text{diag}(\mathbf{A})$ and ν , e.g. $\nu = 2$.
2. Build the **modified** Gauss Newton system:
$$(\mathbf{A} + \lambda \text{diag}(\mathbf{A})) \Delta \mathbf{x} = \mathbf{b}.$$
3. **Solve** it for $\Delta \mathbf{x}$.
4. **Update**
 - If cost is reduced: $\bar{\mathbf{x}}^{k+1} = \bar{\mathbf{x}}^k + \Delta \mathbf{x}$ and expand trust region $\lambda^{k+1} = \lambda^k / \nu$.
 - Else: no update on $\bar{\mathbf{x}}$, contract the trust region $\lambda^{k+1} = \lambda^k \nu$; go to Step 3
5. Check **convergence**: if not OK, go to step 2.

Robust Cost Functions (M-Estimators)

So far, we summed costs $c_i(\mathbf{x}) = \frac{1}{2} \mathbf{e}_i^T \mathbf{R}_i^{-1} \mathbf{e}_i = \rho_2 \left(\underbrace{\sqrt{\mathbf{e}_i^T \mathbf{R}_i^{-1} \mathbf{e}_i}}_{=:r} \right)$, $\rho_2(r) = \frac{1}{2} r^2$.

This does **not account for outliers**:
(we have assumed a Gaussian error distribution)

i.e. outliers will **terribly** distort the result.

Idea: modify, i.e. **robustify** $\rho_2(r) = \frac{1}{2} r^2$.

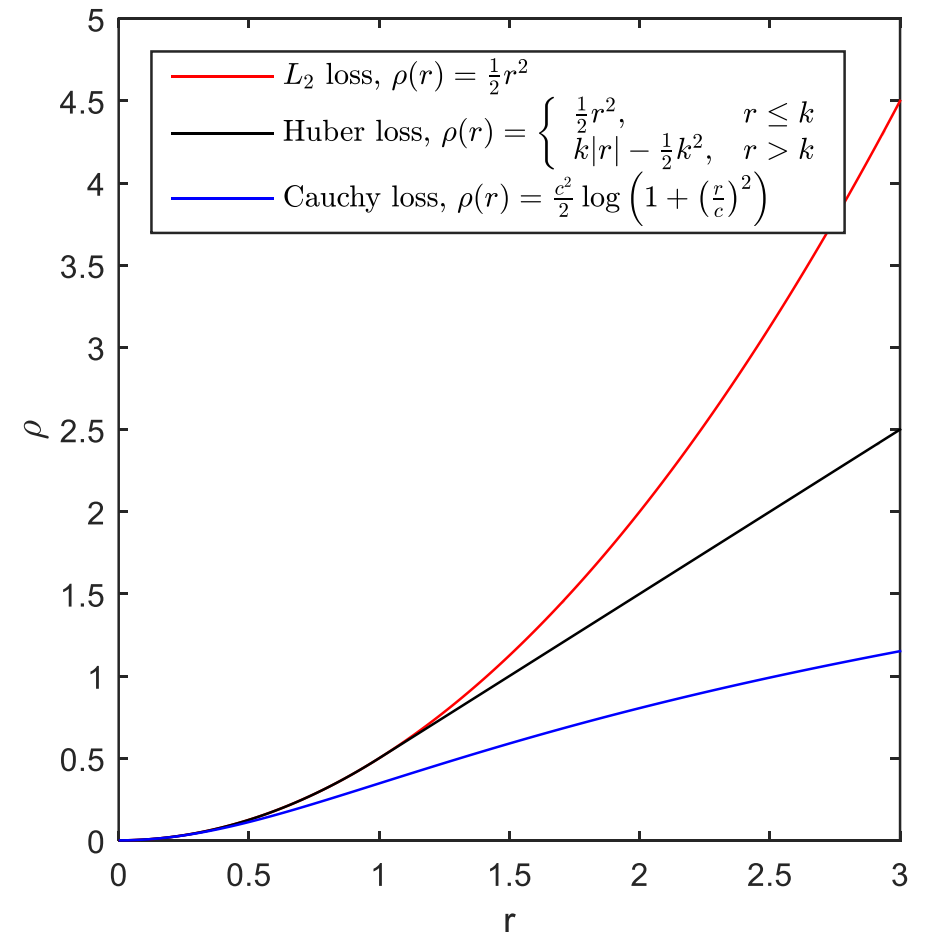
Specifically:

- Modify the error terms $\mathbf{e}'_i = w(r) \mathbf{e}_i$,
- where $w(r) = \frac{\sqrt{2\rho(r)}}{r}$.

Note: when building the normal equations, you will have to apply the chain rule...

...and be careful about small r , but luckily:

$$\lim_{r \rightarrow 0} w(r) = 1, \lim_{r \rightarrow 0} \frac{dw(r)}{dr} = 0$$



Polynomial Fitting and Gauss-Newton Optimization (Python)

Consider the M^{th} order polynomial in the parameter p :

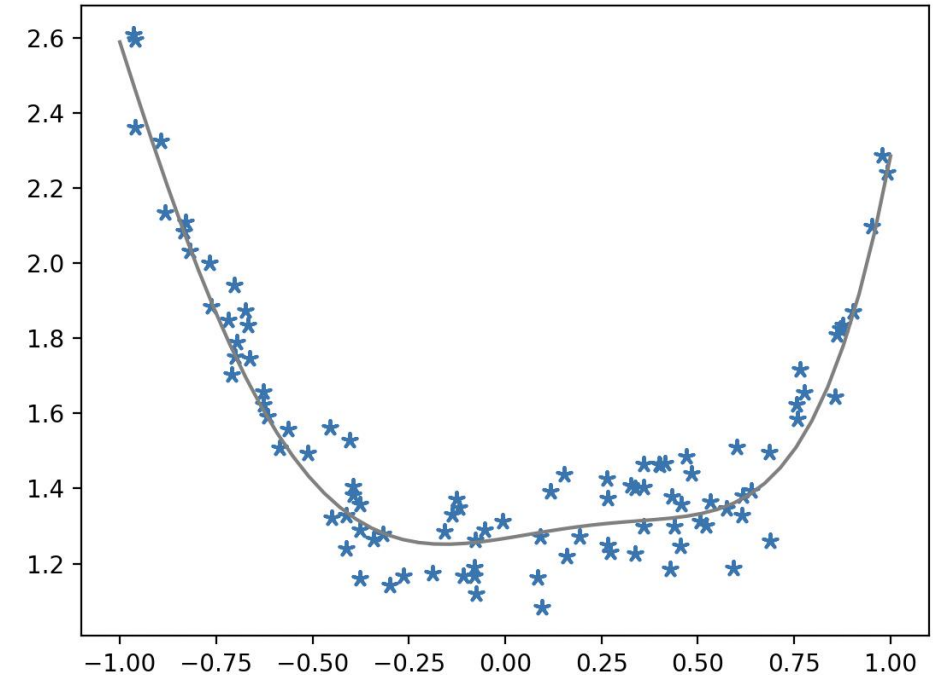
$$z = \sum_{j=0}^M a_j p^j,$$

where you are supposed to estimate the coefficients a_j , i.e. $\mathbf{x} = [a_0, a_1, \dots, a_M]^T \in \mathbb{R}^{M+1}$. We are given a list of N measurements $\tilde{\mathbf{z}} = [\tilde{z}_1, \tilde{z}_2, \dots, \tilde{z}_N]^T$ and corresponding parameters $\tilde{\mathbf{p}} = [\tilde{p}_1, \tilde{p}_2, \dots, \tilde{p}_N]$.

i) With the sum of square errors:

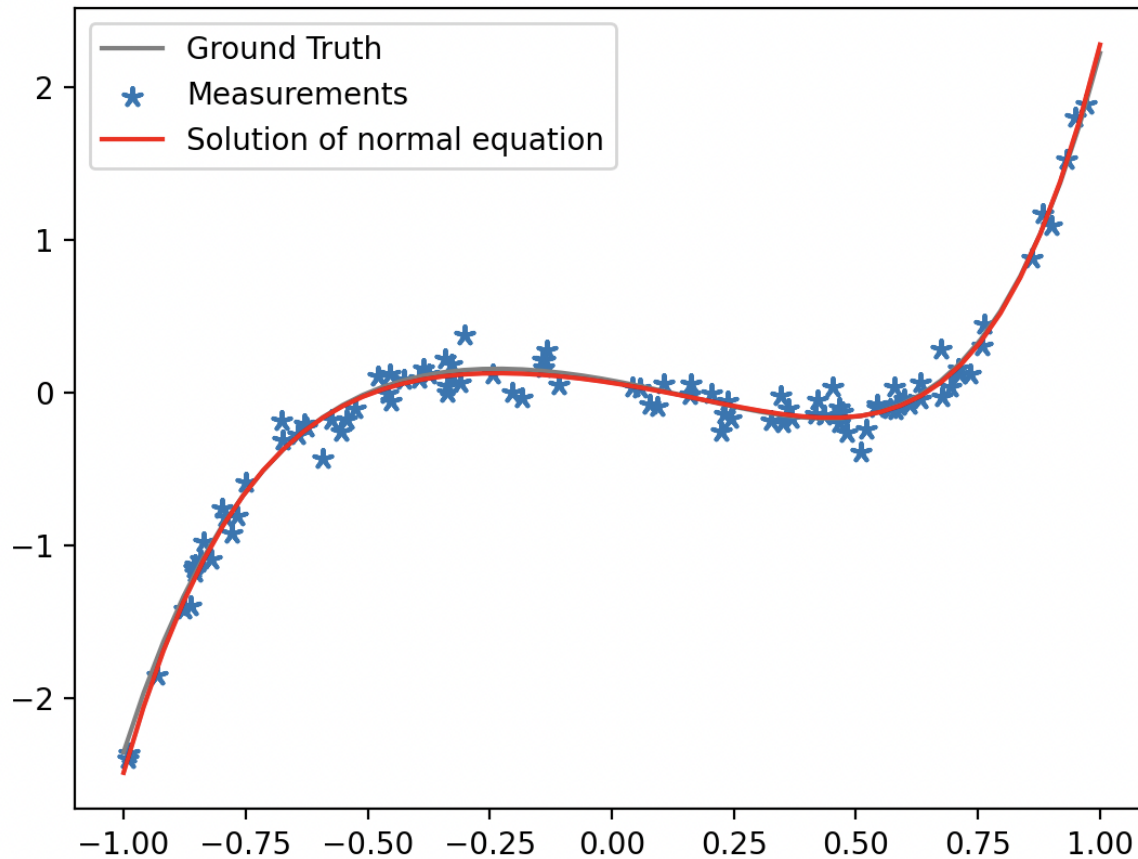
$$c(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N \|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|^2, \quad (1)$$

write down the solution for $\mathbf{x}$ that minimises it.



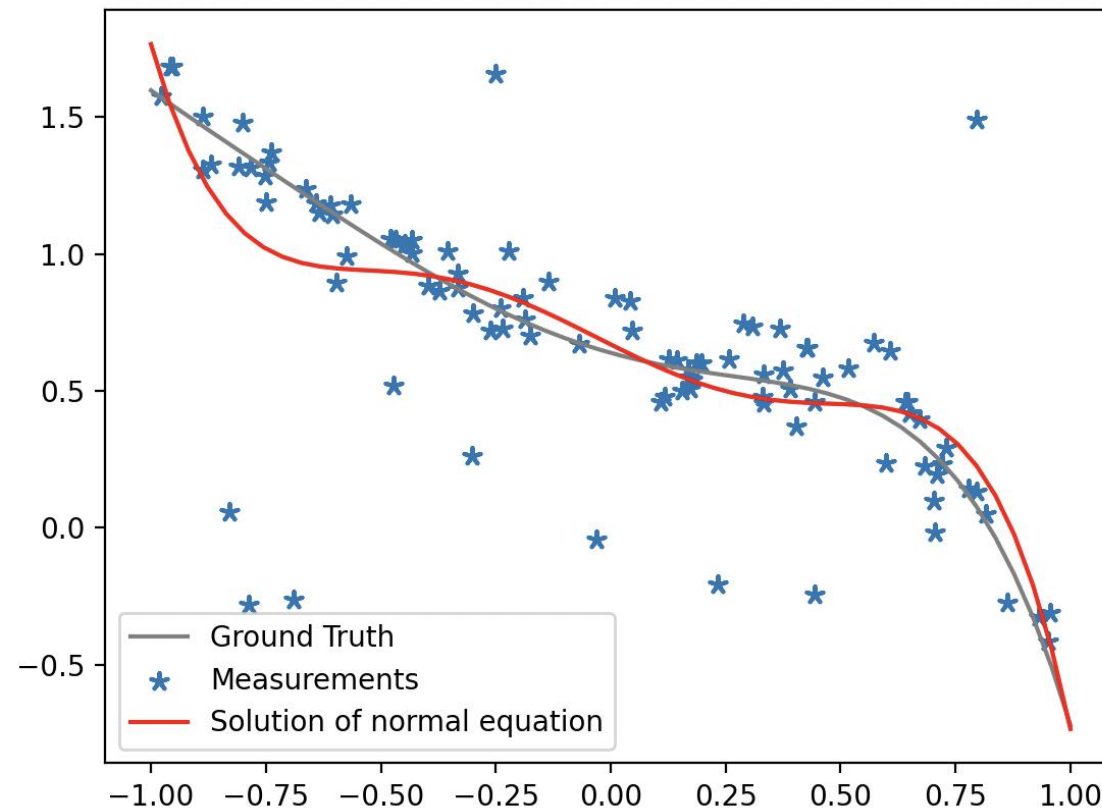
Polynomial Fitting and Gauss-Newton Optimization (Python)

- ii) Implement the above scheme in `e06_least_squares.py`, where the measurements are already provided, and where the implementation should work for any $N > M$.



Polynomial Fitting and Gauss-Newton Optimization (Python)

- iii) Unfortunately, we now have some outlier measurements. Changing the outlier probability parameter P_{outlier} to a value > 0 , observe the effect on your solution.

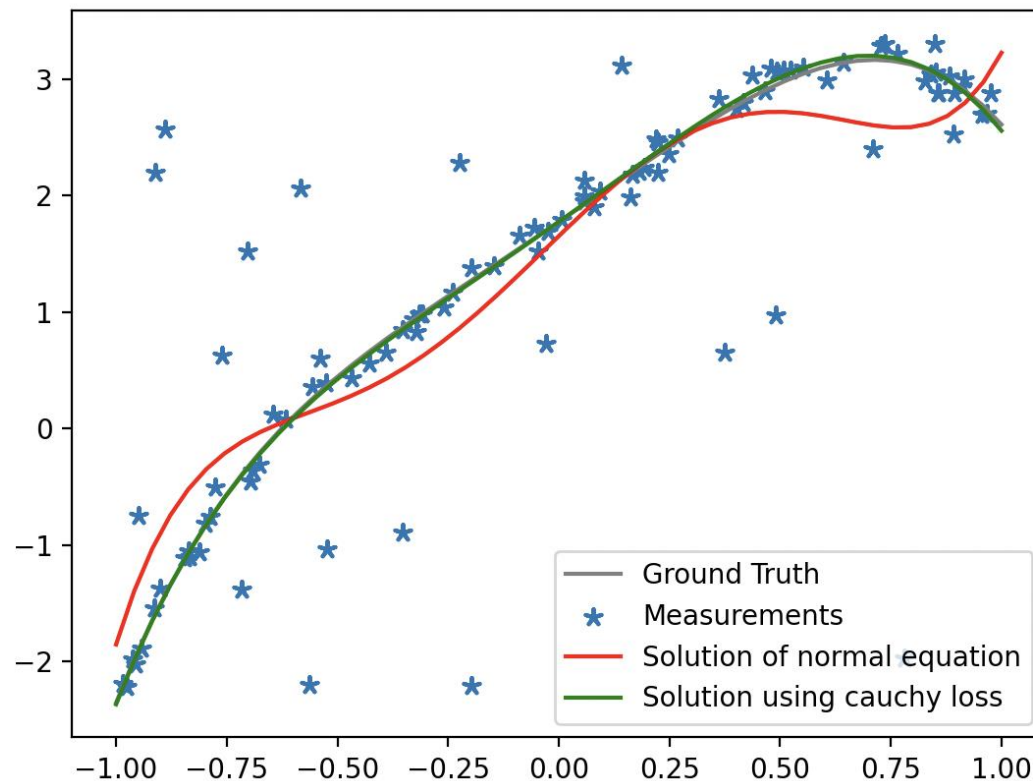


Polynomial Fitting and Gauss-Newton Optimization (Python)

iv) (**Advanced**, Python) To accommodate for outliers, we introduce the alternative of a robust cost function:

$$c(\mathbf{x}) = \sum_{i=1}^N \rho(\|\tilde{z}_i - z(\mathbf{x}, \tilde{p}_i)\|),$$

where $\rho(\cdot)$ stands for a robust cost function. Propose a new implementation to estimate $\mathbf{x}$ using Gauss-Newton. For initialisation you may still use the direct Least Squares Solution from above.



Polynomial Fitting and Gauss-Newton Optimization (Python)

Background: The introduction of a robust cost function makes the error terms of (1) nonlinear, which is why we need to use iterative application of the Normal Equations, i.e. the Gauss-Newton scheme. In order to follow the recipe from Lecture 6, we introduce the error terms $e_i = \tilde{z}_i - h_i(\mathbf{x})$, where of course we are still dealing with a linear measurement function here, i.e. $h_i(\mathbf{x}) = \mathbf{H}_i \mathbf{x}$, with

$$\mathbf{H}_i = \begin{bmatrix} 1 & p_i & p_i^2 & \dots & p_i^M \end{bmatrix},$$

therefore we also get the Jacobian of the error term as $\mathbf{E}_i = -\mathbf{H}_i$.

Now for the robustification, we introduce different functions $\rho(r)$, with $r = \|\mathbf{e}_i\|$. As introduced in the lecture, we can now re-write the problem as

$$c(\mathbf{x}) = \sum_{i=1}^N \rho(\|\mathbf{e}_i\|) = \frac{1}{2} \sum_{i=1}^N \|\mathbf{e}'_i\|^2,$$

with $\mathbf{e}'_i = w(r)\mathbf{e}_i$, and $w(r) = \sqrt{2\rho(r)}/r$.

Typical choices for the robustifier are the Huber Loss ρ_H or the Cauchy Loss ρ_C :

$$\rho_H = \begin{cases} \frac{1}{2}r^2, & r \leq k, \\ k|r| - \frac{1}{2}k^2, & r > k, \end{cases} \quad \rho_C = \frac{c^2}{2} \log \left(1 + \frac{r^2}{c^2} \right),$$

with additional tunable parameters k and c . **Hint:** You will have to derive the new Jacobians $\mathbf{E}'_i$.

2D Motion Tracking System (Python)

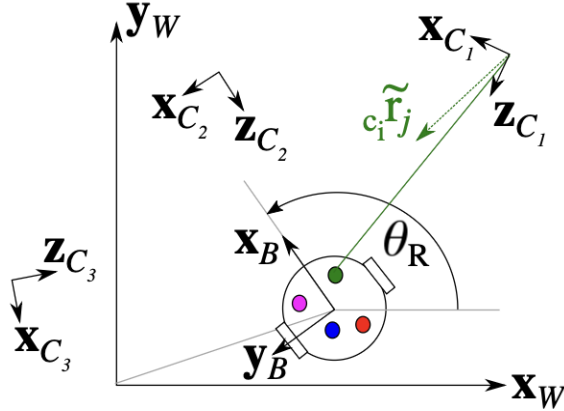


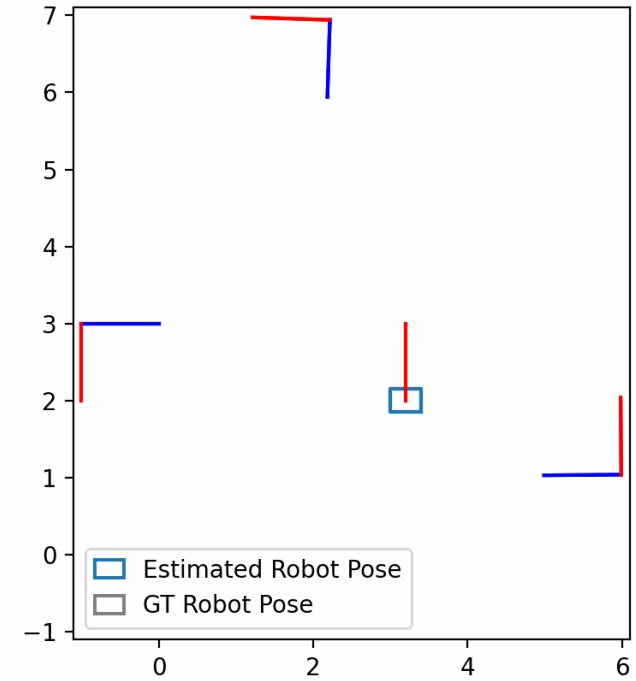
Figure 1: A two-wheel differential drive mobile robot with four fixed markers (indicated by the colorful dots on the robot base) observed by three fixed cameras measuring the bearing vectors $c_i \tilde{\mathbf{r}}_j$.

Consider the two-wheel differential drive mobile robot shown in Figure 1 with the following state:

$$\mathbf{x}^T = [x_R, y_R, \theta_R]^T,$$

where x_R, y_R denote the 2D position of the robot base frame in the world frame, and θ is the orientation of the robot frame relative to the world frame. The robot is equipped with 4 distinct markers (index j) at positions ${}_B \mathbf{t}_{M_j}$ (note the expression in the robot body coordinate frame $\mathcal{F}_B$). Now there are three fixed cameras (index i) located at poses T_{WC_i} that measure the bearing vectors (lengths normalised to 1) $c_i \tilde{\mathbf{r}}_j$ to those markers (2D in the x-y plane). Careful: the bearings are measured relative to the respective camera's individual coordinate frame.

- i) (Python) Open the Python script `e06_diff_drive_mocap.py` to see the set-up: the simulation will show the robot carrying out a 2D motion in a room.



2D Motion Tracking System (Python)

- ii) Formulate the angular difference $e_{i,j}$ between the measured bearing $(C_i \tilde{\mathbf{r}}_j)$ and the actual bearing to the marker j $(C_i \mathbf{t}_{M_j})$ which can be expressed as a function of the state and the known positions of the marker in the body frame, ${}_B \mathbf{t}_{M_j}$. Also write down the respective sum of squares cost function $c(\mathbf{x})$.
- iii) Derive the Jacobian $\mathbf{E}_{i,j} = \frac{\partial e_{i,j}}{\partial \mathbf{x}}$.

2D Motion Tracking System (Python)

- iv) (Python) Using the Gauss-Newton optimisation scheme, implement an estimator for the robot state $\mathbf{x}$ that minimises the above cost $c(\mathbf{x})$ in `e06_diff_drive_mocap.py`.

